

# Exercícios — Aula 07 — Fundamentos de recursão

Técnicas de Programação - 2026/02

Última atualização: September 2, 2026

## Instruções

- Leia o pseudocódigo antes de executar mentalmente.
- Em simulações recursivas, registre chamadas, casos base e retornos.
- Para cada algoritmo, procure responder: qual é o caso base, qual é o passo recursivo e qual é o progresso?

## Exercício 1 — Simulação da pilha de chamadas

Simule o algoritmo para  $n = 4$ .

**Algorithm 1:** SomaAte

**Result:** Executa SOMA-ATE.

**Input** : int  $n$

**Output:** int

```
if  $n = 0$  then
|   return 0
end
resposta  $\leftarrow n + \text{SOMA-ATE}(n - 1)$ 
return resposta
```

Preencha a tabela de chamadas.

| chamada     | ainda precisa calcular | próxima chamada |
|-------------|------------------------|-----------------|
| SOMA-ATE(4) | 4 + SOMA-ATE(3)        |                 |
| SOMA-ATE(3) |                        |                 |
| SOMA-ATE(2) |                        |                 |
| SOMA-ATE(1) |                        |                 |
| SOMA-ATE(0) |                        |                 |

Agora preencha a tabela de retornos.

| retorno de  | valor retornado | cálculo que volta para a chamada anterior |
|-------------|-----------------|-------------------------------------------|
| SOMA-ATE(0) |                 |                                           |
| SOMA-ATE(1) |                 |                                           |
| SOMA-ATE(2) |                 |                                           |
| SOMA-ATE(3) |                 |                                           |
| SOMA-ATE(4) |                 |                                           |

Qual é o resultado final?

## Exercício 2 — Caso base, passo recursivo e progresso

Para cada algoritmo, identifique caso base, passo recursivo e progresso.

---

### Algorithm 2: Contagem

---

**Result:** Executa CONTAGEM.

**Input :** int  $n$

**Output:** none

```

if  $n = 0$  then
    IMPRIMIR("fim")
    return
end
IMPRIMIR( $n$ )
CONTAGEM( $n - 1$ )

```

---



---

### Algorithm 3: Fatorial

---

**Result:** Executa FATORIAL.

**Input :** int  $n$

**Output:** int

```

if  $n = 0$  then
    return 1
end
return  $n * FATORIAL(n - 1)$ 

```

---



---

### Algorithm 4: Dobros

---

**Result:** Executa DOBROS.

**Input :** int  $n$

**Output:** int

```

if  $n = 0$  then
    return 1
end
return  $2 * DOBROS(n - 1)$ 

```

---

| Algoritmo | Caso base | Passo recursivo | Progresso |
|-----------|-----------|-----------------|-----------|
| CONTAGEM  |           |                 |           |
| FATORIAL  |           |                 |           |
| DOBROS    |           |                 |           |

---

Para quais entradas esses algoritmos terminam?

### Exercício 3 — Recursão sem progresso

O algoritmo abaixo tenta somar de 1 até  $n$ , mas está incorreto.

---

**Algorithm 5:** SomaAteComErro

---

**Result:** Executa *SOMA-ATE-COM-ERRO*.**Input** : int  $n$ **Output:** int**if**  $n = 0$  **then**|   **return** 0**end****return**  $n + \textit{SOMA-ATE-COM-ERRO}(n)$ 

---

1. Qual é o caso base?
2. Para  $n = 3$ , qual chamada é feita depois da primeira?
3. Por que essa chamada não aproxima o algoritmo do caso base?
4. Reescreva o algoritmo corrigido.

## Exercício 4 — Potência recursiva

Escreva pseudocódigo para `POTENCIA(base, expoente)`.

Contrato:

- entrada: número `base`, inteiro `expoente`;
- o expoente sempre é maior ou igual a 0;
- saída: `base` elevado a `expoente`.

Exemplos esperados:

- `POTENCIA(2, 0)` retorna 1;
- `POTENCIA(2, 3)` retorna 8;
- `POTENCIA(5, 2)` retorna 25.

Depois de escrever, identifique:

1. caso base;
2. passo recursivo;
3. progresso;
4. número de chamadas para `POTENCIA(2, 4)`.

## Exercício 5 — Laço e recursão

Os dois algoritmos abaixo calculam a mesma soma.

---

**Algorithm 6:** SomaIterativa

---

**Result:** Executa SOMA-ITERATIVA.

**Input** : int  $n$

**Output:** int

$soma \leftarrow 0$

**for**  $i \leftarrow 1$  **to**  $n$  **do**

$soma \leftarrow soma + i$

**end**

**return**  $soma$

---

---

**Algorithm 7:** SomaRecursiva

---

**Result:** Executa SOMA-RECURSIVA.

**Input** : int  $n$

**Output:** int

**if**  $n = 0$  **then**

**return**  $0$

**end**

**return**  $n + \text{SOMA-RECURSIVA}(n - 1)$

---

Para  $n = 5$ , responda:

1. Quantas iterações o algoritmo iterativo faz?
2. Quantas chamadas o algoritmo recursivo faz, contando a chamada com  $n = 0$ ?
3. Qual deles deixa chamadas pendentes na pilha?
4. O crescimento do tempo é constante, linear ou quadrático?
5. O crescimento da pilha na versão recursiva é constante, linear ou quadrático?

## Exercício 6 — Contando chamadas

Para cada algoritmo, determine quantas chamadas acontecem em função de  $n$ .

---

### Algorithm 8: ContaUm

---

**Result:** Executa CONTA-UM.

**Input :** int  $n$

**Output:** int

```

if  $n = 0$  then
  | return 0
end
return  $1 + \text{CONTA-UM}(n - 1)$ 

```

---



---

### Algorithm 9: ContaDois

---

**Result:** Executa CONTA-DOIS.

**Input :** int  $n$

**Output:** int

```

if  $n \leq 0$  then
  | return 0
end
return  $1 + \text{CONTA-DOIS}(n - 2)$ 

```

---

Complete a tabela.

| n | chamadas de CONTA-UM | chamadas de CONTA-DOIS |
|---|----------------------|------------------------|
| 0 |                      |                        |
| 1 |                      |                        |
| 2 |                      |                        |
| 3 |                      |                        |
| 4 |                      |                        |
| 5 |                      |                        |
| 6 |                      |                        |

---

Depois, classifique o custo de cada algoritmo em Big-O.